NEXARION CORE ARCHITECTURAL SPECIFICATION

Understanding Docker & The CI/CD Pipeline

A Deep-Dive Engineering Manual on Containerization, Multi-Stage Builds, Automated Quality Gates, and Production Deployments

---

Executive Overview

Modern multi-agent AI systems combine complex frontend interfaces with heterogeneous backend architectures: Python runtimes, native C-libraries for syntax parsing, language models, database drivers, and frontend bundlers. Without strict infrastructure standardization and automated verification, such systems suffer from environment mismatches, flaky tests, deployment failures, and security vulnerabilities. In **NEXARION**, two foundational pillars guarantee engineering stability:

Pillar 1: Containerization (Docker)	Pillar 2: Automated Verification (CI/CD)
Packages the complete environment (Node.js build tools, Python 3.12, system dependencies like Git and Curl, static assets, and database drivers) into an immutable, self-contained unit that runs identically on any operating system.	A 5-stage automated quality gate in GitHub Actions that validates every commit with linters (Ruff), security scanners (Bandit), matrix integration tests (Pytest on Python 3.11/3.12 with PostgreSQL), frontend unit tests, and Docker build checks before merge.

The Fundamental Problem Being Solved

1. The 'It Works On My Machine' Dilemma: A developer running Windows 11 with Node 18 and Python 3.11 might write code that fails completely when deployed to a Linux server running Python 3.12, due to differing line endings (CRLF vs LF), missing native libraries (e.g. libpq or git), or version discrepancies.

2. Human Verification Bottlenecks: Manually verifying that 64 frontend test suites pass, 7 agent scenarios work, code formatting complies with PEP 8, and no secret keys or security bugs are committed is error-prone. Automated CI/CD replaces manual checks with deterministic guarantees.

3. Zero-Downtime Deployment Confidence: Because production code is packaged into an immutable Docker image verified by automated tests, deployments can happen multiple times a day without fear of breaking running production users.

Core Takeaway: Docker eliminates the *runtime environment problem* by making the execution environment reproducible anywhere. CI/CD eliminates the *code quality & regression problem* by verifying every commit automatically before it can ever touch production.

1. Deep-Dive: Docker Containerization Architecture

Docker packages software into lightweight, standalone executables called **containers**. Unlike heavy Virtual Machines (VMs) that emulate entire operating systems with separate kernels, containers share the host OS kernel while providing isolated user spaces for files, processes, and network interfaces.

The Multi-Stage Build Pattern in NEXARION's Dockerfile

In modern web applications, the frontend (React/TypeScript) requires Node.js, npm, and build toolchains (Vite). However, in production, we only need static HTML, JavaScript, and CSS files! Having Node.js, npm, and 400MB of node_modules in production bloats image size and introduces security vulnerabilities. NEXARION solves this using a **Multi-Stage Dockerfile**:

```
STAGE 1: FRONTEND COMPILATION (node:20-alpine)
  1. WORKDIR /app/frontend
  2. COPY frontend/package.json ./  --> Install npm dependencies
  3. COPY frontend/ ./              --> Copy TypeScript & React source
  4. RUN npm run build              --> Output static files to /app/frontend/dist
STAGE 2: HARDENED PRODUCTION RUNTIME (python:3.12-slim-bookworm)
  5. Install minimal system OS tools: git, curl, ca-certificates
  6. Create dedicated non-root user: appuser (UID 1000, GID 1000)
  7. COPY pyproject.toml README.md ./  --> Install Python backend packages
  8. COPY src/ scripts/ ./             --> Copy backend source code
  9. COPY --from=frontend-builder /app/frontend/dist ./frontend/dist
 10. HEALTHCHECK CMD curl -f http://localhost:8000/health || exit 1
 11. USER appuser                      --> Drop root privileges
 12. CMD uvicorn code_review.web.app:app --host 0.0.0.0 --port 8000
```

Why This Design is Superior:

Metric / Concern	Single-Stage Build (Naive)	NEXARION Multi-Stage (Production)
Final Image Size	~1.2 GB (includes Node, npm, build cache)	~280 MB (75% smaller; only Python & static dist)
Attack Surface	High (Node compilers & root user present)	Minimal (Only runtime Python + non-root appuser)
Build Caching	Rebuilds everything on any file edit	Re-uses dependency layers if package.json hasn't changed
Health Checks	No automated health checks	Native HEALTHCHECK probes /health every 30s

Local Multi-Service Orchestration: Docker Compose

NEXARION uses **Docker Compose** (docker-compose.yml) to orchestrate the application and PostgreSQL database with a single command:

• **Postgres Service (postgres:16-alpine):** Runs isolated DB with healthcheck (pg_isready) and named volume (postgres_data).

• **Backend Service:** Builds the Dockerfile and connects over bridge network using depends_on: { postgres: { condition: service_healthy } }.

2. Deep-Dive: The CI/CD Pipeline Architecture

CI/CD stands for **Continuous Integration** and **Continuous Delivery / Deployment**. Every time code is pushed or a PR is opened on GitHub, NEXARION's workflow (.github/workflows/ci.yml) spins up isolated cloud runners to deterministically validate the changes.

The 5 Automated Quality Gates in NEXARION:

Job Name	Tool & Runner	Exact Purpose & Verification Performed
1. Lint & Format	Ruff (Ubuntu Latest)	Enforces strict PEP 8 formatting rules and syntax standards. Fails immediately if there are unused imports, bad indentation, or undefined variables.
2. Security Scan	Bandit (Ubuntu Latest)	Performs Abstract Syntax Tree (AST) static security analysis. Checks for dangerous functions, SQL injection vectors, hardcoded tokens, and weak crypto.
3. Backend Tests	Pytest + Live Postgres (Python 3.11 & 3.12)	Spawns a real live PostgreSQL 16 container service inside GitHub Actions. Runs all unit, integration, and agent execution tests across Python versions.
4. Frontend Tests	Vite + TypeScript (Node.js 20)	Executes npm run build to check TypeScript types and compile Vite bundle. Runs 64 frontend test suites (modals, chats, routing, toggles).
5. Docker Build	Docker Buildx (GHA Caching)	Builds the entire multi-stage Dockerfile to verify that packaging succeeds and production images can be generated without broken dependencies.

Understanding Matrix Testing & Live Services in CI

Notice in Job 3 (backend-tests), NEXARION uses a **matrix strategy** and a **service container**:

```
strategy:
  matrix:
    python-version: ['3.11', '3.12']   # Runs tests simultaneously on both versions!
services:
  postgres:
    image: postgres:16-alpine           # Spins up actual PostgreSQL in memory
    env:
      POSTGRES_DB: nexarion_test_db
    ports: ['5432:5432']
    options: --health-cmd pg_isready --health-interval 10s
```

Why this matters: Many teams test against SQLite in CI and deploy to PostgreSQL in production. This leads to subtle bugs (different JSON operators, SQL locking, and schema migrations). NEXARION tests against the **exact same PostgreSQL engine in CI** as used in production!

Concurrency Optimization: NEXARION's workflow includes concurrency: { group: ..., cancel-in-progress: true }. If a developer pushes three commits in rapid succession, the pipeline immediately cancels the older builds to save runner minutes and report results on the latest commit faster.

3. End-to-End Production Deployment Lifecycle

Here is the exact lifecycle of how a code commit travels from your machine through Docker and CI/CD to live production:

```
1. LOCAL CODE -> git commit -> git push origin feature-branch
2. GITHUB ACTIONS CI GATEWAY TRIGGERS AUTOMATICALLY:
   - Ruff lints & formats  --> Bandit scans security  --> Pytest tests on Postgres
   - TypeScript compiles    --> Node runs 64 UI tests --> Docker Buildx verifies build
   - If ANY check fails: Merge is BLOCKED. Flawed code never touches production.
3. PULL REQUEST APPROVED & MERGED INTO MAIN
4. CONTINUOUS DELIVERY (CD) WEBHOOK TRIGGER:
   - Render detects merge -> reads render.yaml -> builds Docker image in cloud.
   - Render boots new container -> verifies /health probe -> switches traffic smoothly.
   - Old container is decommissioned. Zero downtime experienced by users.
```

Production Cloud Ecosystem Breakdown

Platform	Component	Role & Integration with Docker / CI
Render	FastAPI Backend & Container Runtime	Reads render.yaml, builds Dockerfile on every push to main, and hosts container on Port 8000.
Neon Tech	Serverless PostgreSQL 16 Database	Provides persistent relational database storage with connection pooling URL (postgresql+psycopg://...).
Vercel	Decoupled React Edge Frontend (Optional)	Connects via BACKEND_URL environment variable, routing API calls and WebSockets to the Render container.
NVIDIA NIM	Cloud AI Inference (Nemotron)	Container injects NVIDIA_API_KEY to orchestrate multi-agent LLM analysis without hardcoding keys.

Developer Cheatsheet: Commands You Can Run Locally

You can run the exact same verification commands locally that the automated CI/CD pipeline runs:

```
# 1. Run linting and code formatting checks locally:
ruff check . && ruff format --check .
# 2. Run security static analysis locally:
bandit -r src -ll
# 3. Run backend unit & integration tests:
pytest --cov=code_review
# 4. Run frontend compilation & test suite:
npm --prefix frontend run build && npm --prefix frontend test
# 5. Build and launch the entire stack in Docker locally:
docker compose up --build -d
```